CHAPTER 9

Asynchronous Jobs with Oban

There's a major tool missing from your Elixir toolbox right now—asynchronous jobs. They are a critical component for scaling up an application. Elixir has big shoes to fill here because Ruby comes with one of the best asynchronous job libraries out there—Sidekiq.

In their most basic form, async jobs run small or large units of work in the background of your application. Once the user has submitted a request, we use async jobs to process additional tasks such as sending emails, performing calculations, or triggering application-specific workflows. With more advanced usage, async jobs are used to power complex business logic with concerns such as uniqueness constraints and rate-limiting.

The first reaction to async jobs that many new Elixirists have is: "don't we have everything we need with GenServer?" It's true that Elixir provides async capabilities out of the box, but libraries provide a lot of functionality that would be complex to build ourselves. We'll talk about one of these libraries—Oban—and some of its capabilities in this chapter.

We'll start by looking at the use cases and implementation concerns that async job libraries handle. You'll understand why job systems are useful and what the traits of a good job system are. Due to Elixir's process model, there are a few different options for how we could handle running async code. We'll go over the different options before finally settling on the Oban[1] library to implement a simple worker in our SMS app.

Let's start by diving into what async jobs are used for.

1. https://hexdocs.pm/oban/Oban.html